



EJERCICIOS

Aplicar lo aprendido

Sumadores RCA / CLA / CSLA + Multiplicadores Shift-Add / Array / Booth

Ejercicio 1 — RCA N – *bit* parametrizable

ENUNCIADO

Implementar en Verilog un Ripple Carry Adder de N bits parametrizable usando full adders estructurales encadenados.

Verificar con un testbench self-checking que cubra:

- borde inferior: $0 + 0$
- borde superior: $\text{max} + \text{max}$
- un caso con carry-out = 1
- 500 casos random

Evaluar el delay aproximado en simulación y graficarlo en función de N para
$$N = \{4, 8, 16, 32\}.$$

DATOS

- N parametrizable (default 8)
- Operandos unsigned
- Carry-in y carry-out explícitos
- Verilog 2001 o SystemVerilog

ENTREGABLES

- ✓ Módulo full_adder.v y rca.v
- ✓ Testbench tb_rca.v con \$display PASS/FAIL
- ✓ Reporte de cantidad de FAs y delay
- ✓ Comentario sobre crítico path observado

Ejercicio 2 — CLA 4-bit → 16-bit jerárquico

ENUNCIADO

Implementar un CLA de 4 bits con las ecuaciones de generate/propagate explícitas:

$$c[i+1] = g[i] \mid (p[i] \& c[i]) \quad p[i] = a[i] \wedge b[i] \\ g[i] = a[i] \& b[i]$$

Luego encadenarlo en grupos de 4 para formar un CLA jerárquico de 16 bits.

Comparar el delay vs un RCA de 16 bits. Verificar con 1000 vectores random.

DATOS

- Bloques de 4 bits encadenados
- Lookahead inter-bloque por carry
- Operandos 16 bits unsigned
- Misma testbench que ejercicio 1

ENTREGABLES

- ✓ Módulo cla4.v + cla16.v
- ✓ Testbench tb_cla.v
- ✓ Tabla delay vs RCA con misma N
- ✓ Discusión sobre dónde gana CLA

Ejercicio 3 — CSLA por bloques de 4 bits(opcional)

ENUNCIADO

Diseñar un Carry Select Adder de 16 bits dividido en 4 bloques de 4 bits.

Cada bloque debe contener:

- Un sumador con carry-in = 0
- Un sumador con carry-in = 1
- Un mux 2:1 controlado por el carry-out del bloque anterior

Comparar área y delay con CLA del ejercicio 2 y discutir cuál sería preferible si el target es $F_{\max}$ alto.

DATOS

- 16 bits totales en 4×4 bloques
- Bloques internos: RCA de 4 bits
- MUX 2:1 entre bloques
- Verificar con misma testbench

ENTREGABLES

- ✓ Módulo cs1a16.v + bloque rca4.v
- ✓ Testbench tb_cs1a.v
- ✓ Tabla comparativa CSLA vs CLA vs RCA
- ✓ Recomendación de arquitectura

Ejercicio 4 — Multiplicador Shift-and-Add (secuencial)

ENUNCIADO

Implementar un multiplicador unsigned de 8×8 bits que produzca el resultado completo en 8 ciclos de reloj.

Componentes:

- Adder de 8 bits
- Registro acumulador de 16 bits
- Shifter (a la izquierda) en cada ciclo
- FSM de control con estados
IDLE → COMPUTE → DONE

Validar con un testbench que compare contra $a*b$ en el host.

DATOS

- Operandos a, b de 8 bits unsigned
- Producto de 16 bits
- Latencia: 8 ciclos
- FSM con start/done

ENTREGABLES

- ✓ Módulo mul_seq.v + control_fsm.v
- ✓ Testbench tb_mul_seq.v con 200 vectores
- ✓ Conteo de ciclos por operación
- ✓ Análisis área vs throughput

Ejercicio 5 — Array Multiplier 8×8 (combinacional)

ENUNCIADO

Implementar un multiplicador 8×8 unsigned combinacional basado en una matriz de full adders.

Estructura:

- Generación de PP con AND-gates
- 7 filas de full adders para reducir
- Una fila final de RCA para el CPA

Medir el delay y comparar contra:

- ① el multiplicador secuencial del Ej. 4
- ② un multiplicador en RTL con "*"

DATOS

- Operandos 8 bits unsigned
- Producto 16 bits
- Matriz de 8×8 PPs
- Sin pipeline (1 ciclo combinacional)

ENTREGABLES

- ✓ Módulo array_mul.v
- ✓ Testbench tb_array_mul.v exhaustivo (256²)
- ✓ Tabla cantidad de FAs y delay teórico
- ✓ Comparativa con secuencial y RTL "*" "

Ejercicio 6 — Booth radix-2 y reducción Wallace/Dadda

ENUNCIADO

Implementar Booth radix-2 para un multiplicador signado 8×8 bits.

Para cada par ($b[i]$, $b[i-1]$) generar:

- 0 si "00" o "11"
- +A si "01"
- -A si "10"

Sumar los PPs con sign-extension adecuada.

Avanzado (opcional): comparar la reducción Wallace vs Dadda en cantidad de HAs/FAs (a nivel conceptual o RTL con CSAs explícitos).

DATOS

- Operandos 8 bits signados (C2)
- Producto 16 bits signado
- $b[-1] = 0$ (asumido)
- Verificación contra $a*b$ en host

ENTREGABLES

- ✓ Módulo booth_r2.v + tb_booth.v
- ✓ Cantidad de PPs generados
- ✓ Tabla comparativa Wallace/Dadda
- ✓ Discusión: cuándo Booth no ayuda